Project Contract

WG Management System (WGMS)

Software Engineering Project – Branching Pipeline Methodology

Team Members: Cemil Aslan, Dzovani Koller, Joan de la Fuente Camps,
Daniel Petriceanu, Taras Trishch, Andrei Ciuta

April 28, 2026

Preamble

This Project Contract serves as the formal technical commitment to realize the **Value Proposition** established in the Project Proposal. Its primary goal is to address the identified sponsor pain points through the specific software functions and architectural assets defined below.

Methodology and Scope Note: The project uses the **Branching Pipeline Methodology**. Core functionality has priority over extended features such as NFC logging, voice automation, LLM-based command parsing, cross-WG communication, and stock prediction.

Contents

1 Team Members & Roles **3**

2 Project Description **3**
2.1 Initial Situation & Existing Similar Products . . . 3
2.2 Project Overview & Technical SE Goals . . . 6
2.3 Target Audience . . . 8

3 Asset Architecture & System Environment **9**
3.1 Asset Network & Infrastructure Stack . . . 9
3.2 Gap & Risk Analysis . . . 10
3.3 C4 Model Architecture . . . 11

4 Requirements **14**
4.1 Functional Requirements (Use Cases) . . . 14
4.2 Scope and Priority Policy . . . 15
4.3 Non-Functional Requirements . . . 16
4.4 Supported Platforms and Optimization Targets . . . 18
4.5 LLM Command Parsing Scope . . . 18
4.6 Iceberg List (Features & Estimation) . . . 19

5 Domain Model **19**
5.1 Domain Model Explanation . . . 21

6 Working Structure & Project Plan **21**
6.1 Distribution of Roles . . . 21
6.2 Horizontal Responsibilities . . . 21
6.3 Project Phases & Milestones . . . 22
6.4 Project Boundaries (Out-of-Scope) . . . 22

7 Supply Components (Deliverables) **23**

8 Risk Assessment and Quality Management **25**
8.1 Risk Matrix . . . 25
8.2 Quality Assurance Strategy . . . 28
8.3 Information Process . . . 28

9 Legal, Compliance and External Hosting Constraints **30**
9.1 Processed Data Categories . . . 30
9.2 GDPR-Oriented Design Principles . . . 30
9.3 User Rights and Data Handling . . . 31
9.4 External Hosting Constraints . . . 31
9.5 Security Constraints . . . 32
9.6 Residual Compliance Risk . . . 32

1 Team Members & Roles

Name	Student ID	Primary Role	Backup Role
Cemil Aslan	12423834	Team Coordinator	Technical Architect
Dzovani Koller	12432196	Technical Architect	Fullstack Developer
Joan de la Fuente Camps	12038098	UI/UX	Technical Architect
Daniel Petriceanu	12320180	Documentation Lead	UI/UX
Taras Trishch	12229192	DevOps	Technical Architect
Andrei Ciuta	12326215	Test Lead	Team Coordinator

2 Project Description

2.1 Initial Situation & Existing Similar Products

Initial Situation and Sponsor Pain Points

A typical shared flat (*Wohngemeinschaft*, WG) in Austria or Germany operates as a small informal organisation: rent and utilities are pooled, groceries are bought ad hoc by whoever is at the supermarket, chores rotate by spoken or hand-written agreement, and the financial liability for the household is concentrated on a single main tenant (*Hauptmieter*in*) who holds the lease. The tooling that supports this organisation, however, is fragmented across at least four parallel channels: a WhatsApp group for communication, a printed schedule on the fridge for chores, an expense-splitting app such as Splitwise for receipts, and the memory of the main tenant for everything else. Receipts disappear into chat history, paper schedules decay within weeks, and verbal agreements are forgotten between the people who made them and the people who join the household later.

The recurring failure modes of this arrangement constitute the sponsor pain points the WGMS project commits to address:

- **Financial leakage on the main tenant.** Micro-expenses below approximately five euros are systematically not reclaimed because the social cost of asking exceeds the monetary value. Over an eleven-month academic year, the cumulative under-recovery is non-trivial and is borne entirely by the lease holder.
- **Unbounded coordination overhead.** Monthly *Kassensturz* (manual reconciliation) requires the main tenant to scroll weeks of WhatsApp history, cross-reference receipts, and individually chase three to five roommates. The opportunity cost is several hours of work per month that produces no output beyond restoring an accurate balance.
- **Liability without authority.** The main tenant carries legal responsibility toward the landlord and economic responsibility toward third parties, but has no enforceable mechanism to ensure that roommates discharge their share. Disputes therefore escalate to the level of personal relationships rather than being resolved at the tooling layer.
- **Onboarding gap on roommate turnover.** When a roommate joins or leaves mid-term, the verbal and paper-based record of the household state cannot be transferred. The new roommate inherits an opaque set of conventions and is routinely surprised by their first overdue chore or first unexpected expense, eroding goodwill from day one.

Existing Similar Products

Three categories of solution currently address fragments of the problem above. None addresses it as a whole.

Splitwise is the dominant English-language expense-splitting service, available on the web, iOS, and Android. Its strength is a mature settlement model that handles unequal splits, multiple currencies, and partial payments. With respect to the WG use case it is purely a financial ledger: it has no concept of recurring chores, shopping lists, household membership, or shared communication. WGs that adopt Splitwise must continue to maintain WhatsApp, paper schedules, and any shopping coordination outside the tool.

Flatastic (also marketed as *Die WG-App*) is the closest direct competitor in the German-speaking market. It is a mobile-first application that bundles four modules: a cleaning schedule with a points-based ranking system, a synchronised shopping list, a shared expense ledger, and a pinboard called *Shouts* for lightweight intra-WG communication. Flatastic is offered as a freemium product with a subscription that unlocks features such as expense export, photo attachments on receipts, and per-item shopping notes. Flatastic has a substantial installed base and a long product history, but is constrained in three ways relevant to WGMS. First, its primary surface is mobile, which excludes desktop-first administrative workflows. Second, it treats every household uniformly — there is no domain-level distinction between a financially-liable main tenant and an ordinary roommate. Third, it has no concept of inter-WG interaction.

Splid represents a third class of competitor: lightweight, single-purpose group expense trackers (alongside Tricount and similar apps). These are excellent for tracking one trip or one project but do not provide the persistent, role-aware household model that a long-lived WG requires.

Status quo: WhatsApp and paper. The actual incumbent that WGMS must displace is the combination of a WhatsApp group, a fridge-mounted cleaning schedule, and the ad-hoc memory of whoever paid last. This combination has the lowest possible onboarding cost (zero) and the highest possible familiarity. Any specialised tool must demonstrably outperform it on the workflows that matter most — adding an expense, viewing the current balance, and confirming that a chore is done — to be adopted at all.

Capability	**WhatsApp + paper**	**Splitwise**	**Splid**	**Flatastic**	**WGMS**
Shared expense tracking	–	✓	✓	✓	✓
Cascading debt settlement engine	–	✓	partial	✓	✓
Chore / cleaning schedule	paper	–	–	✓	✓
Shared shopping list	–	–	–	✓	✓
One-click expense from shopping	–	–	–	–	✓
History-aware fair chore balancing	–	–	–	partial	✓
Consumable stock prediction	–	–	–	–	✓
Persistent household role model	–	–	–	–	✓
Tenant lifecycle (join / leave)	–	–	–	partial	✓
NFC / passive appliance logging	–	–	–	–	✓
Cross-WG events & social feed	–	–	–	–	✓
Web-first, no install required	–	✓	–	partial	✓
GDPR-aligned, EU-hostable	n/a	partial	partial	✓	✓

Table 1: Feature coverage of WGMS against the most relevant existing solutions. *Partial* indicates that the capability exists in a degraded or limited form (e.g. a chore module without history-aware balancing).

Identified Gap and Unique Selling Propositions

Table 1 identifies a concrete market gap: no existing product offers a role-aware, web-first, integrated WG operating system. WGMS positions itself against this gap through five unique selling propositions:

- **Single source of truth for the entire WG lifecycle.** Expenses, chores, shopping, communication, and appliance usage live in one persistent application with one coherent domain model. The main tenant gains a single dashboard that answers "what is open?" without context-switching between four tools.
- **Role-aware household with seamless onboarding.** WGMS is the only system in the comparison that distinguishes the financially-liable Main Tenant from ordinary Roommates as first-class domain concepts, enforced at the API layer through role-based access control. Onboarding via time-limited invitation links transfers the entire household state to a new roommate at the moment they join, eliminating the "opaque inheritance" failure mode described above.
- **Deterministic settlement engine.** Cascading debts are reduced via a minimum-transactions debt-graph algorithm executed inside a Spring `@Transactional` boundary using `BigDecimal` arithmetic with fixed HALF_UP rounding to two decimal places. This eliminates the floating-point drift that plagues hand-rolled spreadsheet settlements and is verifiable through unit tests, which makes it directly gradeable.
- **Passive cross-platform NFC appliance logging.** NFC tags are encoded as deep-link URLs that the mobile OS resolves natively into the Angular SPA — no Web NFC API required. The user taps an appliance and the application opens on the corresponding logging view. This works identically on iOS Safari (which does not expose Web NFC) and on Android, and is the project's most distinctive non-CRUD feature.
- **Cross-WG social layer.** The optional event and feed module allows registered WGs to interact with one another — inviting another flat to a party, following a friend's WG. No competitor offers this, and it is the one module that creates a network effect rather than a closed-loop tool.

Business Model and Monetisation Paths

The Project Contract scope does not include payment processing, ad serving, or B2B contracting. The architecture is, however, designed to be compatible with the following commercial paths, recorded here to motivate design decisions (the role model, the optional modules, the GDPR-aligned data handling) without expanding the implementation scope. None of these paths produces a deliverable in this contract or influences grading.

- **Freemium with non-invasive advertising.** The core workflows — registration, WG creation, expenses, chores, shopping, dashboard — remain free and ad-supported through placements modelled on the New York Times Games approach: a single static banner per session, no interstitials, no interruption of task flows. A small subset of quality-of-life features (e.g. data export, advanced filters) may later move behind an ad-free subscription. The workflows that drive adoption are never paywalled, ensuring parity with the zero-cost incumbent (WhatsApp + paper).
- **Micro-fee on settlement transactions.** A small fixed fee in the order of one to five euro cents per recorded settlement could be levied to monetise the deterministic settlement engine without burdening micro-expenses themselves. The fee would be capped per

month and waived in the freemium tier above a configurable threshold, so users who settle once a month pay nothing measurable.

- **B2B partnership with property management firms (Immobilienverwaltungen).** Property management firms that lease student housing inventory absorb the indirect cost of tenant disputes through support overhead, escalations to the landlord, and reduced retention. A licensing arrangement allows such firms to offer WGMS as a pre-installed amenity for their tenants, with the firm paying a per-WG fee and the residents receiving the ad-free experience by default. The partnership also opens two firm-side workflows that the existing data model can support without migration: residents reporting damages through the app (linked to the household and the appliance domain), and the firm dispatching invoices to the WG through the same interface used for shared expenses. A future portfolio role for the firm would be a strictly read-only aggregate view across its WGs — no leakage between flats.
- **Student-targeted sponsorships.** A WG is, in aggregate, a small commercial cohort with a predictable life cycle. Targeted sponsorship slots could be offered to partners whose products fit that cohort: banks promoting student accounts, local event services such as Goodnight.at or Freyzeit.at, and the kind of merchant offers found in printed *Gutscheinhefte*. Sponsorship slots would be served through the same banner placements as the freemium tier, never inside task flows, and would be configurable per WG so that households can opt out of categories they find irrelevant.

All four paths are listed in Section 6.4 as commercial options that the system is designed to *not preclude*. The role-aware data model, the household-scoped privacy boundary, and the GDPR-aligned data handling are the technical preconditions for any of them.

2.2 Project Overview & Technical SE Goals

Project Overview

WGMS is a responsive, browser-based information system that consolidates the operational coordination of a shared household into a single application. It is delivered as an Angular Single Page Application backed by a Java Spring Boot REST API and an embedded H2 relational database, deployed via GitLab CI/CD to the university's Kubernetes cluster (the technology stack is fully specified in Section 3.1). Functionally, the system is organised around seven use case packages (UC-01 to UC-07, Section 4.1) covering registration and household creation, task and cleaning management, expense tracking and settlement, collaborative shopping, intra- and inter-WG communication and events, the central dashboard, and the optional NFC-based appliance log.

The product is intentionally scoped as a *web application only*. No native iOS or Android clients are produced. Cross-device compatibility is achieved through a responsive design tested on viewports between 360 pixels and 1920 pixels in width, and through the passive NFC NDEF URL routing strategy described in Section 3.2, which avoids any platform-specific NFC API. The system handles real shared-household data (names, email addresses, expenses, messages, appliance usage records) but does not execute monetary transactions, integrate with banking systems, or process special categories of personal data under the GDPR.

Technical Software Engineering Goals

The Project Contract commits the team to the following measurable technical goals. Each goal is phrased so that fulfilment can be assessed directly from the GitLab repository and the

deployed artefact, in line with the assessment criteria communicated by the SEPR teaching team.

- **TG-01 — Three-tier architecture.** A clean separation between Angular SPA, Spring Boot REST API, and persistence layer (Spring Data JPA over H2). No business logic in the frontend; no database access outside repositories.
- **TG-02 — Authentication and authorisation.** Stateless authentication via JSON Web Tokens, password hashing via BCrypt, and role-based access control enforced on the backend (not only hidden in the frontend). Two roles are distinguished at the domain level: *Main Tenant* and *Roommate*.
- **TG-03 — Deterministic financial logic.** The settlement engine reduces the inter-roommate debt graph to its minimum-transactions form using `java.math.BigDecimal` with HALF_UP rounding to two decimal places, executed within a Spring `@Transactional` service boundary. Settlement correctness is verified by unit tests covering equal splits, unequal splits, rounding edge cases, deleted-user fallbacks, and partially-settled payments (NFR-07, NFR-10).
- **TG-04 — API documentation and contracts.** All REST endpoints are described by an OpenAPI (Swagger) specification served by the running application. The Angular client consumes this contract, and integration tests verify that documented and actual behaviour agree.
- **TG-05 — Continuous integration and deployment.** Every push to a feature branch triggers a GitLab pipeline that builds the project, runs the full test suite, and (on the main branch) builds a Docker image, pushes it to the project registry, and deploys it to the university Kubernetes cluster. No deployment artefact is produced manually.
- **TG-06 — Cross-platform NFC integration without a native app.** Physical NFC tags are encoded as deep-link URLs pointing to specific appliance routes in the Angular SPA. The integration is verified to work on iOS Safari (where the Web NFC API is not exposed) and on Chromium-based Android browsers. A QR-code fallback is provided for devices without NFC support, satisfying NFR-12.
- **TG-07 — GDPR-aligned data handling.** The system supports user-initiated account deletion or anonymisation, a data export endpoint for self-service access, time-limited invitation tokens, and an explicit data inventory documented in Section 10.1. No production secrets are committed to the repository.
- **TG-08 — Browser and viewport support.** The application targets the current and previous major releases of Chrome, Firefox, Safari, and Edge as of project start, on desktop and mobile viewports between 360 and 1920 pixels wide. Browser support is recorded in the README and verified through cross-browser smoke tests before each milestone.
- **TG-09 — Test coverage on critical logic.** Backend services classified as critical (authentication, settlement engine, task assignment, invitation handling) are covered by automated tests. The Quality Assurance Strategy (Section 9.2) defines the merge criteria.
- **TG-10 — Maintainability and handover.** The repository contains a project README, an architecture overview consistent with the C4 diagrams in Section 3.3, and documentation for every public module. The handover package follows the deliverables list in Section 7.

The non-CRUD differentiators of the system — the cascading settlement engine (TG-03), the passive NFC URL routing (TG-06), the history-aware chore balancing (FR-15), the consumable stock prediction (FR-16), and the cross-WG social feed (FR-21) — are the features the team

will demonstrate as the substantive engineering contribution of the project, beyond the standard create-read-update-delete surface.

2.3 Target Audience

The product is aimed at two primary user roles, plus a sponsor and two secondary audiences relevant to architectural decisions but not implemented in the version covered by this contract.

Project Sponsor

For the purpose of the SEPR course, the Project Sponsor is the teaching team of the *Software Engineering Project* module at TU Wien, represented by the assigned assistant. The sponsor's acceptance criterion is the fulfilment of this Project Contract: the implementation of the high- and medium-priority features in Section 4.3, the architectural commitments in Section 3, and the quality and risk commitments in Sections 9 and 10. Grading is based on the contents of the GitLab repository at each Milestone Review (MR1–MR3) and on the demonstrated working software.

In a hypothetical commercial deployment of the same system the *de facto* sponsor would be the Main Tenant role described below, who carries the financial liability of the household, derives the largest share of value from the tool, and is the role most likely to introduce WGMS into a flat and to advocate for its adoption by the other roommates.

Persona 1: The Main Tenant

Name and role. Anna Berger, 26, master's student in Business Informatics at TU Wien, main tenant of a four-person WG in the 9th district of Vienna for the past two years.

Context. Anna signed the rental contract with the landlord and pays the consolidated rent, electricity, and internet bills from her account. Her three roommates transfer their shares back to her each month. Two of them are reliable; one is chronically late.

Goals. Recover every euro that the WG owes her without an awkward conversation. Spend less than thirty minutes per month on household administration. Hand over an accurate record to a new roommate when one moves out.

Frustrations. Receipts that disappear into the WhatsApp group. The cumulative cost of three- and four-euro items she has stopped asking about. The end-of-month *Kassensturz*, which currently consumes between three and five hours. Awkwardness when she has to "remind" a friend who is also a roommate about a debt.

Quote. "I shouldn't have to be the WG's accountant just because my name is on the lease."

How WGMS helps Anna. The dashboard surfaces every open balance and overdue task at a glance (FR-12). The settlement engine resolves cascading debts deterministically (FR-05, FR-06). Push and email reminders (FR-11) replace the awkward in-person conversation. The history view (FR-13) gives her a defensible audit trail when a roommate disputes a charge.

Persona 2: The Roommate

Name and role. Lukas Stein, 22, third-year bachelor's student in Computer Science, joined Anna's WG six months ago after the previous tenant moved out.

Context. Lukas moved into an existing household mid-semester. The cleaning rotation, the shared shopping conventions, and the previous balance owed to the main tenant were all communicated to him verbally on move-in day. He has retained roughly half of it.

Goals. Pull his weight without having to ask repeatedly what is expected. Pay his share of expenses promptly so that he is not the one being chased. Avoid being the WG member that others complain about behind his back.

Frustrations. The WhatsApp group has more than a thousand messages from before he joined and is impossible to catch up on. He missed the first cleaning rotation because nobody told him about it. He has paid for groceries twice and forgotten to claim the second one.

Quote. "I'm happy to pay my share — I just need someone to actually tell me what it is."

How WGMS helps Lukas. On joining the WG via a time-limited invitation link (FR-02), Lukas immediately sees his current share of open expenses, his assigned chores, and the active shopping list (FR-12). Push notifications (FR-11) flag a new task or expense without him having to monitor the chat. The one-click expense generation (FR-14) makes claiming a shared purchase frictionless enough that he actually does it.

Secondary Audiences

Two further audiences are not user-facing roles in the version of WGMS covered by this contract, but they motivate design choices that the implementation honours from the first milestone onward.

The External Guest. A user who is a member of a different WG and interacts with Anna's flat exclusively through the optional cross-WG social feed (FR-21) and through joining cross-WG events (FR-18). The External Guest never has access to the host WG's expenses, tasks, or messages. This audience exists in the documentation specifically to anchor the privacy boundary tested by NFR-03 and NFR-04: every public-facing feature is designed against the question "would an external guest see this, and is that intended?"

Property Management Firms. Firms that lease student housing inventory hold an indirect stake in the disputes WGMS aims to reduce: tenant escalations, landlord complaints, and turnover all carry support cost. No property-management role is implemented in this version, but the role-aware data model and the GDPR-aligned data handling (Section 10) make a future strictly read-only portfolio role technically feasible without data-model migration. This is the audience that motivates the B2B commercial path described in Section 2.1, and the design constraint it imposes — strict isolation between WGs by default — is honoured throughout the implementation.

3 Asset Architecture & System Environment

An asset is defined as any component, framework, or infrastructure that contributes value to the information system. The WG Management System (WGMS) architecture follows the provided university baseline and extends it with domain-specific modules for household management, NFC usage logging, voice-assisted automation, and LLM-based command parsing.

3.1 Asset Network & Infrastructure Stack

To fulfill the domain requirements while adhering to the technical directives, the project uses the following technology stack:

- **Client-Side (Frontend):** The system is implemented as a responsive Single Page Application (SPA) using **Angular 21.x**. Node.js **24.x LTS** is used for package management, development tooling, and production builds. The UI is optimized for the defined reference devices and browser versions listed in the compatibility section. The goal is not pixel-perfect support for every device, but a stable and usable experience on the agreed target platforms.
- **Server-Side (Backend):** The backend is a **Java Spring Boot 4.0.x** REST API targeting **OpenJDK 26** in the reference setup. It uses *Spring Security* for Role-Based Access Control (RBAC) and authentication via JSON Web Tokens. It also uses *MapStruct* for DTO mapping and *OpenAPI/Swagger* for interactive API documentation.
- **Data Persistence:** For the university project scope, the system uses an embedded **H2 Database 2.4.x** in automatic mixed mode. Data access is abstracted using **Spring Data JPA** and the Jakarta Persistence API (JPA). In deployed demo environments, the database file should either be stored on a persistent volume or recreated through documented seed data. WGMS does not claim production-grade database durability with embedded H2. A migration path to PostgreSQL is documented as future work.
- **Infrastructure & CI/CD:** The project uses **GitLab CI/CD**. After a successful pipeline run, the deployment job builds a Docker image, pushes it to the project registry, and deploys it to the university's **Kubernetes Cluster** where applicable. If the deployment environment is limited, the minimum deliverable is a reproducible local Docker setup and documented deployment instructions.
- **Optional AI-powered ASR System:** Voice-assisted automation is treated as an extended feature. The architecture reserves an optional integration point for a standalone **Whisper-Flow** container based on Python/FastAPI. If implemented, the Angular frontend streams raw audio via WebSockets to this service. The service performs in-memory transcription using local Whisper models and returns text to the Spring Boot backend for command parsing and validation. If time or device constraints prevent real-time audio implementation, written natural-language commands may be delivered first, while real-time speech transcription remains future work.
- **Optional Google Cloud Vertex AI Gemini Command Parser:** ASR output or typed natural-language commands are parsed by a backend-only Google Cloud Vertex AI / Gemini integration. The reference model is `gemini-3-flash-preview` (Gemini 3 Flash). Because Gemini 3 Flash is currently provided as a preview model, the exact model ID is kept configurable in the backend environment configuration. The LLM is limited to command-to-JSON parsing, receives only minimal context, and never executes actions directly. Backend validation, permission checks, and user confirmation remain the planned safety mechanism.

3.2 Gap & Risk Analysis

Because WGMS is developed on top of the provided baseline, the implementation gap consists of extending the existing vertical slice with WG-specific domain logic.

The first major challenge is the **Cross-Platform NFC Integration**. Bridging physical household appliances to a web application is technically complex because iOS Safari does not support the active Web NFC API through JavaScript. To mitigate this constraint and avoid a native mobile app, the architecture uses *Passive NDEF URL Routing*. NFC tags are encoded with deep-link URLs pointing to specific application resources. When a user taps a smartphone against an appliance tag, the operating system opens the corresponding WGMS view where supported.

This feature remains optional and should not block the core WGMS workflow if device support is limited.

The second critical risk area is the implementation of the **Settlement Engine**. Calculating shared expenses and cascading debts between roommates requires precise decimal handling and safe transaction boundaries. This risk will be addressed through Spring's `@Transactional` service layer and automated tests for the financial calculation logic.

The third risk involves the **H2 Database Lifecycle**. Since H2 may run in a local or containerized environment, data persistence across restarts must be handled carefully. Test data, database initialization, and migration scripts need to be documented to avoid accidental data loss. The project does not claim production-grade database durability with embedded H2.

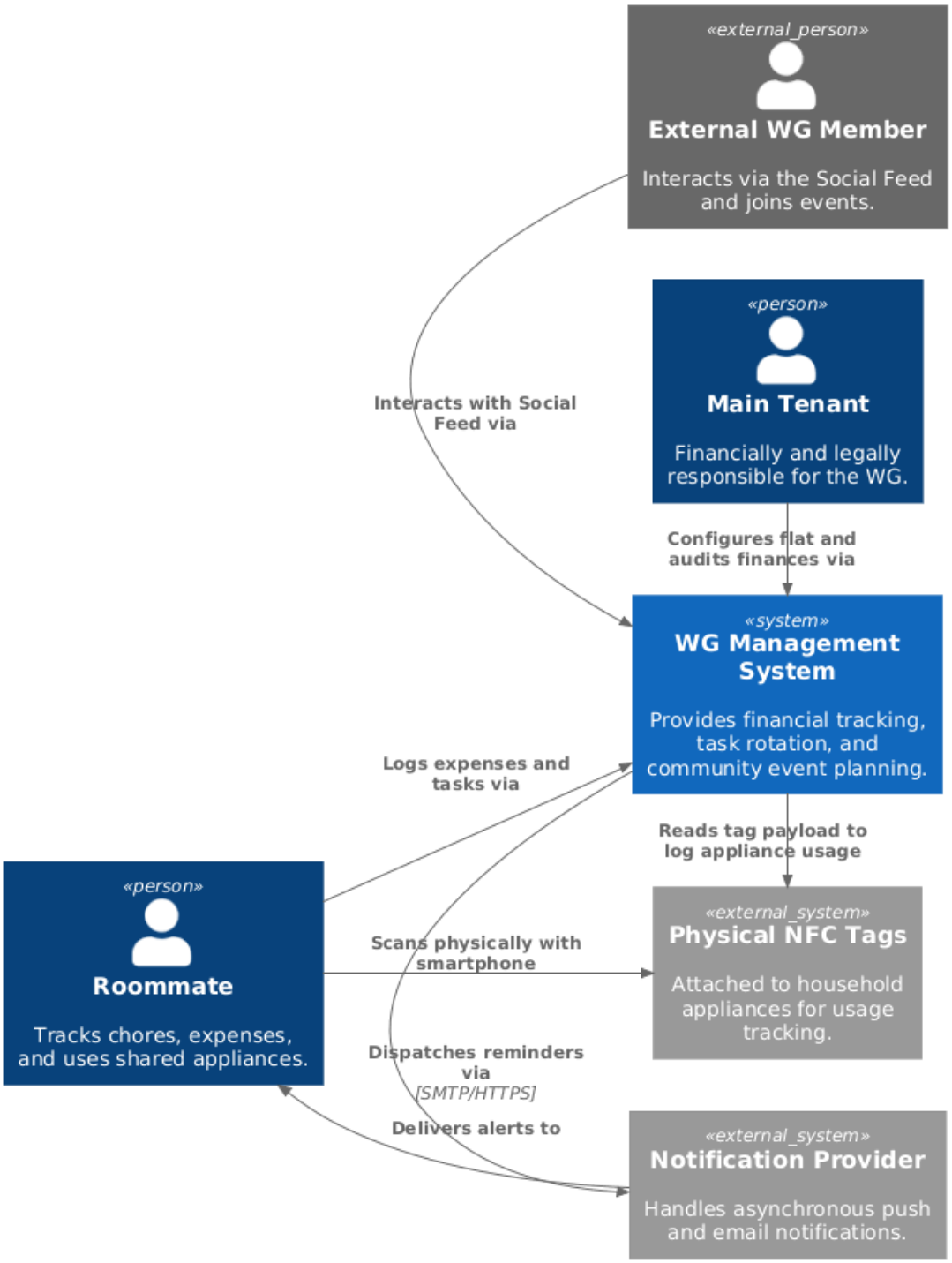
The fourth risk is the **Voice-Assisted Automation Pipeline**. Audio transcription and command parsing may produce incorrect results. Therefore, the system restricts automation to predefined intents, validates parsed JSON against a schema, and asks for confirmation before sensitive changes such as expenses, task completion, appliance logging, or payment-related updates are executed.

The fifth risk is the **LLM Command Parser**. Because LLM outputs can be incomplete, malformed, or semantically wrong, the LLM is not allowed to directly execute system actions. It only proposes a structured JSON command. The backend validates the JSON schema, checks user permissions, verifies referenced entities, and asks for user confirmation for sensitive changes before execution.

3.3 C4 Model Architecture

To visually bridge business requirements and technical implementation, the architecture is modeled using the C4 approach.

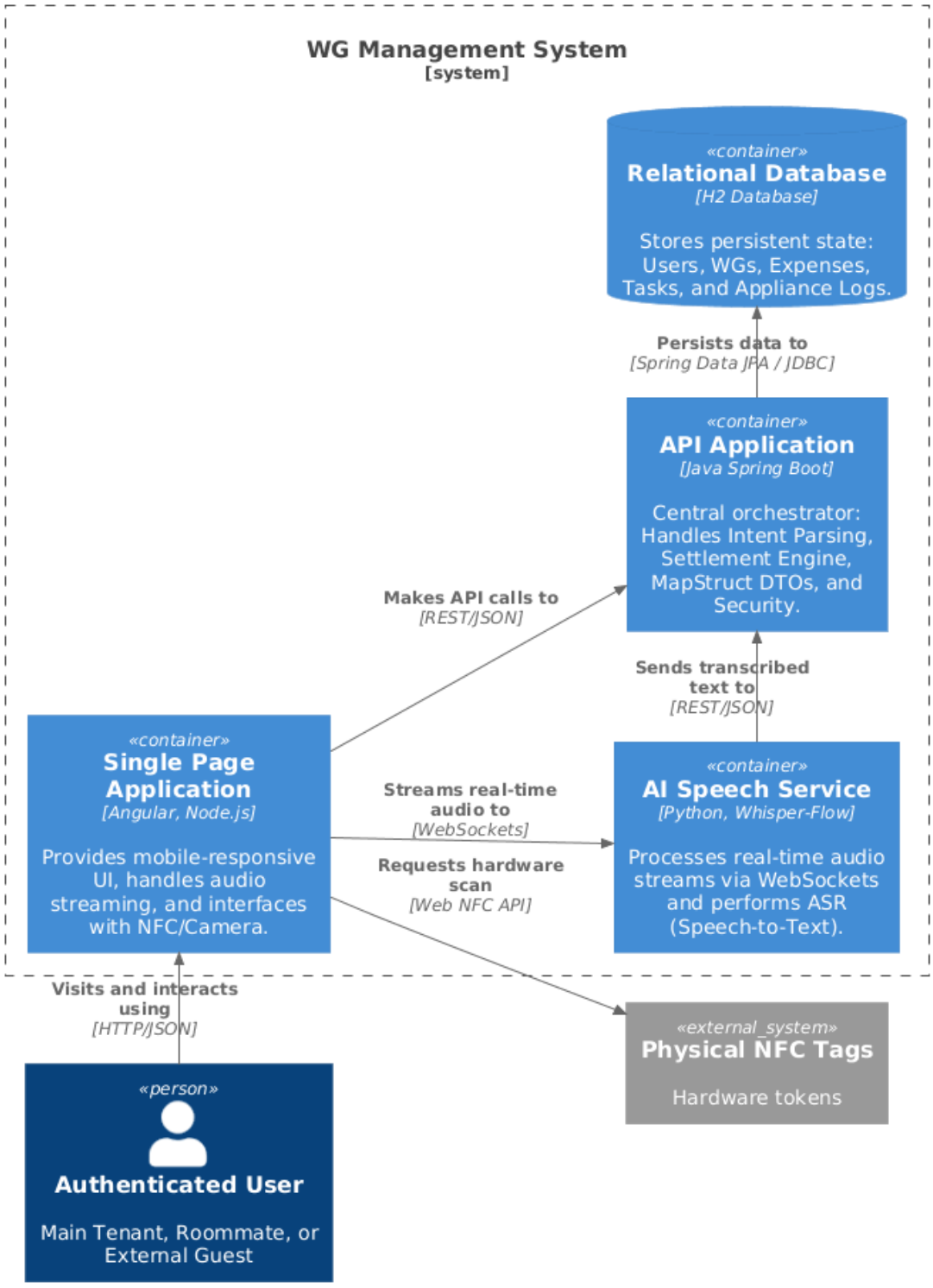
Level C1: System Context
The context diagram illustrates WGMS within its broader environment. The core actors are the *Main Tenant* and the *Roommate*. The system interfaces with physical NFC tags attached to appliances and may use a notification provider for asynchronous reminders. Voice-assisted commands are handled through the WGMS frontend and internal ASR/LLM command-parsing pipeline.



Level C2: Container Diagram
Zooming into the system boundary, the WGMS decomposes into the following primary and optional containers:

- **Angular Web Application:** Delivers the responsive user interface. It handles normal user interaction, NFC deep links, typed natural-language commands, and audio input for voice-assisted automation where implemented.
- **Spring Boot API Service:** Acts as the central orchestrator. It contains authentication, authorization, the settlement engine, household management logic, appliance manager, notification logic, command validation, and command execution logic.

- **H2 Database:** Stores persistent application state such as users, households, tasks, expenses, shopping items, messages, appliance logs, automation records, LLM-parsed command results, and confirmation decisions.
- **Optional Whisper-Flow Speech Service:** Auxiliary Python/FastAPI service for real-time speech-to-text transcription. It processes audio streams in memory and returns transcribed text for command parsing. This container is part of the extended architecture. If not fully implemented, written natural-language command input remains the fallback path.
- **Optional Google Cloud Vertex AI Gemini Integration:** External Google Cloud Vertex AI Gemini model endpoint used to transform ASR transcripts or typed natural-language commands into a predefined JSON command schema. It does not execute actions, does not replace backend validation, and does not provide open-ended chatbot functionality.



4 Requirements

4.1 Functional Requirements (Use Cases)

The functional scope of the WG Management System (WGMS) contains the following core use case packages. They are categorized by priority to distinguish core functionality from extended functionality.

1. **UC-01: User Registration and WG Initialization** (Priority: High)

Allows users to register securely and create a new shared household (WG). The creator can invite roommates through time-limited invitation links.

2. **UC-02: Cleaning Schedule and Task Management** (Priority: High)
   Roommates can create, assign, and track recurring household tasks or cleaning duties.
3. **UC-03: Shared Expense Tracking and Settlement** (Priority: High)
   Allows roommates to enter shared expenses, split costs, track balances, and monitor pending versus settled payments.
4. **UC-04: Collaborative Shopping Management** (Priority: High)
   Enables users to add personal or shared shopping requests, which can be aggregated into a household shopping list.
5. **UC-05: WG Communication and Event Planning** (Priority: Medium)
   Supports internal communication through direct or group chats, event planning, and optional social feed interaction with other registered WGs.
6. **UC-06: Central Dashboard and Notifications Management** (Priority: High)
   Provides an aggregated dashboard showing open responsibilities, payment status, history, and active reminders.
7. **UC-07: NFC-based Appliance Log** (Priority: Low)
   Users can access appliance-specific logging views through NFC tags to record usage of shared household resources. This feature is treated as extended functionality and is not required for core project success.
8. **UC-08: Voice-Assisted and Natural-Language Household Automation** (Priority: Low)
   Enables users to interact with the system through typed or spoken household commands such as “I bought milk for 3 euros” or “Mark the kitchen cleaning as done”. If speech input is available, the system transcribes the audio first. The resulting text is then parsed by a Google Cloud Vertex AI Gemini command parser into a predefined JSON schema. The backend validates the schema, checks permissions and referenced entities, shows the interpreted action to the user, and applies the update only after validation or confirmation.

4.2 Scope and Priority Policy

To keep the project feasible within the available effort of 720 hours, the project follows a strict priority policy.

Core functionality is prioritized before extended functionality is implemented. The core scope consists of registration and login, WG creation and invitations, task and cleaning schedule management, shared expense tracking, settlement overview, collaborative shopping management, dashboard functionality, history views, security basics, and documentation.

Extended functionality includes NFC appliance usage tracking, voice-assisted automation, Google Cloud Gemini command parsing, direct chat, cross-WG chat, public feed functionality, stock prediction, fair chore balancing, and one-click expense generation. These features are implemented only after the core workflow is stable. If time is limited, extended features may be delivered as simplified prototypes, documented design concepts, or future-work items.

The system is considered successful if the core WG-management workflow is functional, tested, documented, and demonstrable on the defined reference devices. Extended features improve the final result but should not endanger the delivery of the core system.

4.3 Non-Functional Requirements

In addition to the functional requirements, WGMS is designed to satisfy the following non-functional requirements. They are especially important because the system processes personal data, internal household communication, financial records, automation commands, LLM parser outputs, and appliance usage histories.

ID	Category	Requirement	Verification
NFR-01	Security / Authentication	Users must authenticate before accessing WG-specific data. Passwords must never be stored in plain text and must be protected using secure hashing.	Code review, security checklist, login tests
NFR-02	Authorization / RBAC	Access to sensitive functions must depend on the user's role. For example, only the Main Tenant or WG administrator may manage invitations, remove members, or change important WG settings.	Role-based access tests
NFR-03	Data Privacy	The system should only collect data required for WG management purposes, such as account data, task assignments, expenses, messages, notification preferences, automation records, and command-parser logs.	Data inventory review
NFR-04	GDPR Support	Users must be able to request access to their stored data, correction of incorrect data, and deletion or anonymization of their account data where technically and legally possible.	Manual compliance test cases
NFR-05	Performance	Standard dashboard pages should load within 2 seconds under normal project demo conditions with up to 8 roommates, 100 tasks, 100 expenses, and 100 shopping items. Lists such as expenses, tasks, messages, and feed posts should support pagination or lazy loading if they grow large.	Performance smoke tests
NFR-06	Reliability	Core data such as expenses, payments, tasks, WG membership, and automation action logs should not be lost during normal demo usage. Database initialization and migration scripts should be documented and tested. In demo deployments, either persistent storage or reproducible seed data should be provided.	Integration tests, migration tests, demo restore test
NFR-07	Financial Accuracy	Expense splitting and settlement calculations should use precise decimal handling. Rounding must be deterministic and visible to the user.	Unit tests for settlement logic

NFR-08	Usability	Main workflows such as adding an expense, checking assigned tasks, viewing open payments, and confirming an automation action should be completable without external instructions by a peer tester.	Usability review, peer testing
NFR-09	Maintainability	The codebase should follow a clear separation between frontend, backend, database access, business logic, and optional ASR/LLM automation components. Important modules must be documented sufficiently for handover.	Code review, architecture review
NFR-10	Testability	Critical backend services, especially authentication, task assignment, expense tracking, settlement logic, command parsing validation, and automation execution, should be covered by automated tests.	CI/CD pipeline results
NFR-11	Auditability	Important actions such as creating expenses, settling payments, assigning tasks, changing WG membership, parsing automation commands, and executing automation actions should be traceable through timestamps and responsible users.	Database inspection, functional tests
NFR-12	Compatibility	The system is optimized for the defined reference platforms: iPhone 12 Pro with iOS 26.x/Safari, Google Pixel 7 with Android 16/Chrome 147.x+, and desktop Chrome 147.x or Firefox Stable at 1366x768 px resolution or higher. Other modern devices are supported on a best-effort basis. NFC-based features are optional and not required for core workflow completion.	Reference-device tests, responsive layout tests
NFR-13	Automation Safety	ASR and LLM command-parser outputs are validated against a predefined JSON schema before execution. Sensitive actions such as expense creation, payment updates, task completion, or appliance usage logging should require explicit user confirmation. If no ASR implementation is completed, the same validation flow applies to written natural-language commands.	JSON schema tests, confirmation-flow tests

NFR-14	Voice Data Privacy	Raw audio streams should be processed only temporarily for transcription and should not be stored permanently. Stored automation data should be limited to command text, parsed intent, structured payload, validation result, confirmation decision, status, and timestamps.	Privacy review, log inspection
NFR-15	LLM Data Minimization	The Google Cloud Vertex AI Gemini parser is designed to receive only the minimum information required to parse the command. It should not receive passwords, JWT tokens, private message histories, full financial histories, raw audio files, or unrelated WG data.	Prompt review, log inspection, code review
NFR-16	LLM Output Control	The LLM output is treated as untrusted input. The backend rejects malformed JSON, unknown intents, unauthorized actions, invalid entity references, and actions outside the predefined household-management scope.	Parser tests, negative test cases, backend validation tests

4.4 Supported Platforms and Optimization Targets

To avoid uncontrolled optimization effort, WGMS defines a limited set of reference platforms. Other devices are supported only on a best-effort basis and are not guaranteed to be pixel-perfect.

Category	**Reference Device**	**OS / Browser**	**Target Viewport**	**Support Level**
Primary iOS Mobile	iPhone 12 Pro	iOS 26.x / Safari	390 $\times$ 844 CSS px	Optimized and tested
Primary Android Mobile	Google Pixel 7	Android 16 / Chrome 147.x Stable or newer	412 $\times$ 915 CSS px	Optimized and tested
Small Mobile Boundary	Generic smartphone	Modern iOS/Android browser	Minimum 360 CSS px width	Should remain usable
Desktop Reference	Standard laptop display	Chrome 147.x or Firefox Stable	1366 $\times$ 768 px and higher	Optimized and tested

Active Web NFC, tablets, foldables, smart TVs, and unusual embedded browsers are outside the dedicated optimization scope. NFC appliance logging is tested only if implemented as an extended feature. Voice input is an extended demo feature; written natural-language input and normal forms are the fallback.

4.5 LLM Command Parsing Scope

The LLM is used because ASR output is plain text and must be converted into a structured user-action schema. WGMS uses Google Cloud Vertex AI / Gemini only for predefined command

parsing, for example `CREATE_EXPENSE`, `ADD_SHOPPING_ITEM`, `COMPLETE_TASK`, `LOG_APPLIANCE_USAGE`, `CREATE_EVENT`, `SEND_WG_MESSAGE`, or `UNKNOWN_OR_UNSUPPORTED`. The expected output is a strict JSON object containing intent, confidence, entities, confirmation requirement, and a user-facing summary. The backend validates the schema, permissions, WG membership, and referenced entities before showing a confirmation screen or rejecting the command.

4.6 Iceberg List (Features & Estimation)

The following table maps planned features to use cases, priorities, planned milestone versions, effort estimates, and responsible feature leads. The total estimated effort is **720 hours**, structured as **120 hours per team member** across the six team members.

ID	Feature / Responsibility	Use Case	Priority	Category	Version	Effort	Lead
FR-01	User registration and login	UC-01	High	Core	V1	24h	Dzovani Koller
FR-02	Secure WG creation & invitations	UC-01	High	Core	V1	32h	Dzovani Koller
FR-03	Task creation and assignment	UC-02	High	Core	V2	24h	Joan de la Fuente C.
FR-04	Cleaning schedule management	UC-02	High	Core	V2	32h	Joan de la Fuente C.
FR-05	Shared expense tracking	UC-03	High	Core	V2	48h	Cemil Aslan
FR-06	Payment status overview	UC-03	High	Core	V3	32h	Cemil Aslan
FR-07	Shopping list management	UC-04	High	Core	V2	32h	Daniel Petriceanu
FR-08	Targeted shopping request collection	UC-04	Med	Core	V2	24h	Daniel Petriceanu
FR-09	Aggregated household shopping list	UC-04	Med	Core	V3	40h	Daniel Petriceanu
FR-10	Shopping item suggestion/voting	UC-04	Med	Extended	V4	24h	Daniel Petriceanu
FR-11	Notifications and reminders	UC-06	Med	Extended	V4	40h	Taras Trishch
FR-12	Dashboard with current responsibilities	UC-06	High	Core	V3	40h	Joan de la Fuente C.
FR-13	History of tasks and payments	UC-06	High	Core	V3	32h	Andrei Ciuta
FR-14	One-click expense generation	UC-03	Med	Extended	V4	40h	Cemil Aslan
FR-15	Fair chore balancing via history	UC-02	Low	Extended	V5	40h	Andrei Ciuta
FR-16	Consumable stock prediction	UC-04	Low	Extended	V5	32h	Taras Trishch
FR-17	NFC-based appliance usage tracking	UC-07	Low	Extended	V5	24h	Dzovani Koller
FR-18	Event creation and planning	UC-05	Med	Extended	V4	32h	Andrei Ciuta
FR-19	WG group and cross-WG chat	UC-05	Med	Extended	V5	24h	Taras Trishch
FR-20	Direct chat between roommates	UC-05	Low	Extended	V5	16h	Andrei Ciuta
FR-21	Following WGs and public feed	UC-05	Low	Extended	V5	16h	Dzovani Koller
FR-22	Real-time audio streaming UI	UC-08	Low	Extended	V5	20h	Joan de la Fuente C.
FR-23	Google Cloud Gemini command parsing, JSON validation & entity extraction	UC-08	Low	Extended	V5	28h	Dzovani Koller
FR-24	Self-hosted Whisper-Flow deployment	UC-08	Low	Extended	V5	24h	Taras Trishch
Total (Target: 120h/person $\times$ 6 members)						**720h**	

Table 4: Iceberg List of Features, Use Case Mappings, and Estimation

5 Domain Model

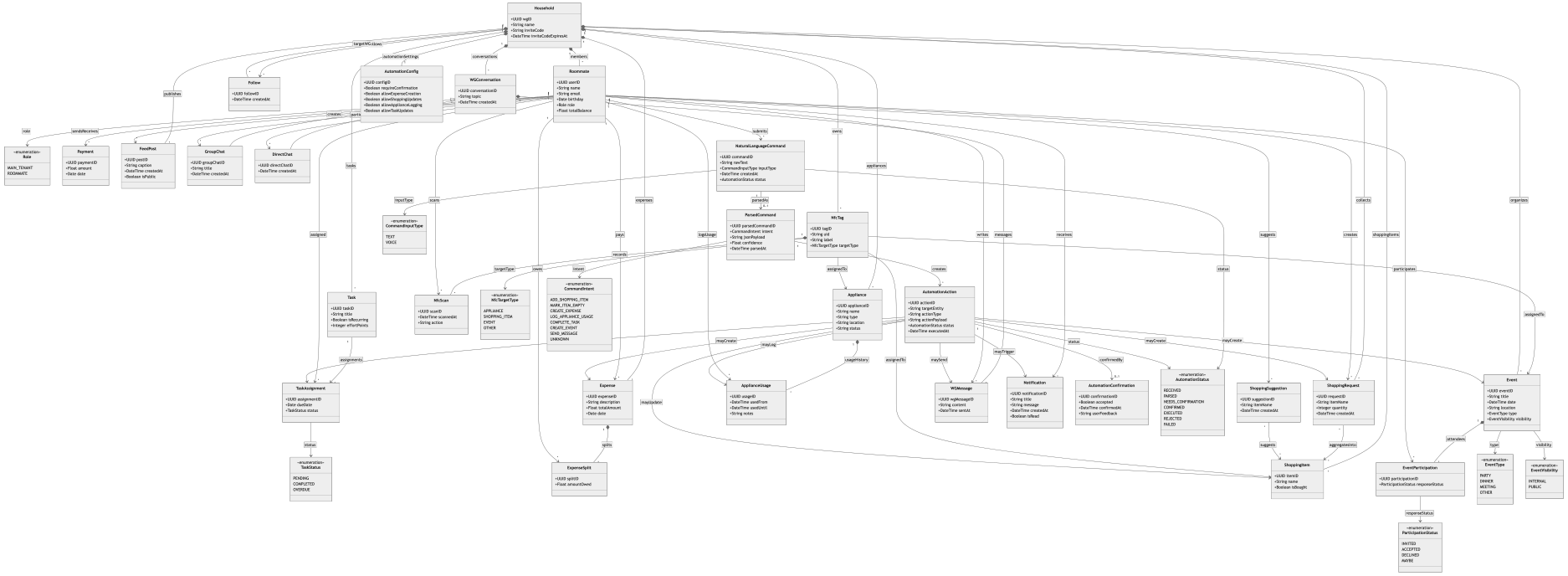


Figure 1: Detailed domain model in landscape format. Click here to view in detail and zoom.

5.1 Domain Model Explanation

The domain model describes the main data structure of the WG Management System. The central entity is `Household`, which represents one shared flat and connects the main system areas: roommates, tasks, expenses, shopping, appliances, events, communication, notifications, and automation.

A `Roommate` belongs to a household and has a role such as `MAIN_TENANT` or `ROOMMATE`. Tasks are modeled with `Task` and `TaskAssignment`, where assignments store the due date and status of a task. The finance area consists of `Expense`, `ExpenseSplit`, and `Payment`, allowing the system to calculate balances and track settlements between roommates.

Shopping is represented by `ShoppingItem`, `ShoppingRequest`, and `ShoppingSuggestion`. Appliance usage is modeled through `Appliance`, `NfcTag`, `NfcScan`, and `ApplianceUsage`. Communication and social features are represented by `WGConversation`, `WGMessage`, `GroupChat`, `DirectChat`, `Event`, `EventParticipation`, `FeedPost`, and `Follow`.

The model also includes a natural-language automation layer. Users can enter or speak commands such as “I bought milk” or “I used the air fryer”. Spoken commands are first transcribed into text by the ASR service where implemented. The resulting text is stored as a `NaturalLanguageCommand` object and parsed by the Google Cloud Vertex AI Gemini parser into a structured `ParsedCommand`. The system then creates `AutomationAction` objects, which can update shopping items, create expenses, log appliance usage, complete tasks, create events, send messages, or trigger notifications. `AutomationConfig` defines whether confirmation is required before automated actions are executed.

Some features, such as the dashboard, history view, fair chore balancing, stock prediction, and one-click expense generation, are modeled as derived behavior instead of separate entities because they are calculated from existing domain data.

6 Working Structure & Project Plan

6.1 Distribution of Roles

The project work is distributed according to the primary roles assigned in the team structure. Each team member is mainly responsible for tasks that match their role while still supporting other areas when necessary.

- **Team Coordinator:** Organizes project planning, milestone tracking, and task distribution.
- **Technical Architect:** Designs the system architecture and supports backend/frontend implementation decisions.
- **UI/UX:** Designs user interfaces and ensures usability across the application.
- **Documentation Lead:** Maintains project documentation and prepares submission materials.
- **DevOps:** Supports deployment processes and technical infrastructure tasks.
- **Test Lead:** Responsible for testing activities and quality assurance.

6.2 Horizontal Responsibilities

In addition to their primary feature responsibilities, team members are responsible for several cross-functional tasks throughout the project lifecycle.

- **Project Coordination (Cemil Aslan):** Organizes meetings, tracks milestone progress, manages task distribution, and ensures deadlines are met.
- **Architecture Oversight (Dzovani Koller):** Maintains consistency between frontend, backend, database, ASR, and LLM automation architecture decisions.
- **UI/UX Consistency (Joan de la Fuente Camps):** Ensures a consistent user interface design and usability across all application modules and reference devices.
- **Documentation Management (Daniel Petriceanu):** Maintains project documentation, reports, diagrams, and submission materials.
- **Testing and Quality Assurance (Andrei Ciuta):** Oversees testing processes, bug tracking, and verifies feature stability before merges.
- **Deployment and Technical Support (Taras Trishch):** Monitors deployment functionality, resolves infrastructure-related issues, and supports integration processes.

6.3 Project Phases & Milestones

Timeline	Milestone	Deliverables
Week 1–2	**MS1 / V1: Project Setup Preparation**	Initial group meeting, introduction to the tutor, and discussion and selection of the project topic. Preparation and delivery of the PV presentation, as well as preparation and delivery of the PA presentation. Setup of GitLab and introduction of time-tracking structures.
Week 3–4	**MS2 / V2: Project Setup**	Fully configured project setup with working user registration/login and successful user-to-WG connection. Backend, frontend, H2 database, CI skeleton, and initial API documentation are available, together with initial work on core household management systems.
Week 5–6	**MS3 / V3: Core Household Management and Feature Expansion**	Working system for managing tasks, cleaning responsibilities, expenses, and shopping requests. Main Tenant and Roommate roles are enforced on the backend. Complete core feature set with dashboards, payment status overview, history views, and improved user visibility. Core workflows are tested on the defined reference devices.
Week 7–9	**MS4 / V4: Extended Features**	Extended functionality such as notifications, events, one-click expense generation, shopping suggestions, and selected communication features. Extended features should not destabilize the core workflow.
Week 9–11	**MS5 / V5: Finalization**	Final version of the application with planned core features implemented, tested, and documented. Extended features such as NFC logging, ASR, Google Cloud Gemini parsing, cross-WG chat, and stock prediction are delivered as implemented features, prototypes, or documented future work depending on remaining project capacity.

6.4 Project Boundaries (Out-of-Scope)

- **No Native App:** The project will only provide a responsive web application and not separate Android or iOS applications.
- **No Real Money Transfers:** The system tracks expenses, balances, and payment status, but does not handle actual money transfers or banking integrations.

- **No Full Accounting System:** WGMS does not provide tax reports, invoices, legally binding financial statements, or advanced bookkeeping features.
- **No Smart Home Control:** NFC features are only used for logging appliance usage. The system will not control household devices or connect to IoT platforms.
- **No Guaranteed NFC Support:** NFC-based features are treated as optional extended functionality and may be reduced, limited, or documented as future work if device support is insufficient.
- **No Fully Autonomous Sensitive Automation:** Automation actions affecting expenses, payments, task status, appliance logs, or important household records are planned to require validation and user confirmation.
- **No General-Purpose Chatbot:** Natural-language input and the Google Cloud Vertex AI Gemini parser are limited to predefined household-management commands and do not support open-ended conversation.
- **Controlled External LLM Dependency:** The project intentionally uses Google Cloud Vertex AI / Gemini as the external LLM provider for command parsing. The integration is backend-only, documented as an external AI processor, and limited to predefined household-management commands. Additional or alternative LLM providers are out of scope unless separately reviewed.
- **No Persistent Raw Audio Storage:** Raw voice recordings are not part of the persistent data model and should only be processed temporarily for transcription.
- **No Legal Rental Management:** WGMS supports household organization but does not replace rental contracts, landlord communication, or legal tenant agreements.
- **No Universal Device Optimization:** The UI is optimized for the defined reference devices and viewport sizes. Other modern devices are supported on a best-effort basis, but the project does not guarantee pixel-perfect layouts for all smartphones, tablets, foldables, or desktop resolutions.
- **No Tablet or Foldable-Specific UI:** Tablets, foldable phones, smart TVs, and unusual embedded browsers are outside the dedicated optimization scope.
- **No Browser-Specific Native Features:** Features that depend on limited browser APIs, such as active Web NFC, should not block the core workflows if unavailable.
- **No commercial monetisation infrastructure:** payment rocessing for premium subscriptions, ad serving, micro-fee collection, B2B billing portals, and sponsorship marketplaces are out of scope. The architecture is designed to be compatible with the monetisation paths described in Section 2.1 but the implementation produces no commercial deliverable.

7 Supply Components (Deliverables)

The delivery is split into core deliverables and optional extended deliverables so that grading can focus on a stable core workflow while still allowing advanced features.

Core deliverables are the Angular web application, Spring Boot REST API, H2 persistence layer, authentication and authorization, WG membership, task and cleaning schedule management, expense and settlement module, shopping module, dashboard/history views, CI/CD configuration, deployment/setup documentation, API documentation, C4 diagrams, UML model, and test documentation.

Extended deliverables are NFC appliance logging, communication/event features, Google Cloud Gemini command parsing, ASR-based voice input, Whisper-Flow deployment, fair chore balancing, stock prediction, shopping suggestions, and one-click expense generation. These are implemented after the core system is stable and may be delivered as working features, prototypes, or documented future work.

The delivery explicitly excludes native mobile apps, real payment processing, smart-home control, legal rental management, formal accounting, persistent raw audio storage, direct frontend access to Gemini, uncontrolled external LLM providers, and a general-purpose chatbot.

8 Risk Assessment and Quality Management

8.1 Risk Matrix

The following risk matrix identifies the most important technical, organizational, legal, and adoption-related risks for the WGMS project. Each risk is assessed according to its probability and impact. Countermeasures are defined to reduce either the probability of the risk occurring or the damage caused if the risk occurs.

ID	Risk	Probability	Impact	Countermeasure
R-01	Low adoption by roommates because users continue using WhatsApp or informal communication instead of WGMS.	Medium	High	Keep the main workflows faster than using a chat message. Focus on dashboard visibility, reminders, and simple input forms. Introduce the system through the Main Tenant as the central sponsor.
R-02	Scope creep due to many extended features such as cross-WG chat, public feed, NFC tracking, voice automation, LLM parsing, and stock prediction.	High	High	Apply strict prioritization. Core features such as registration, WG creation, tasks, expenses, shopping lists, dashboard, and history are prioritized before extended features. Extended features may be reduced to prototypes or documented as future work. This makes the stable core workflow clear for assessment while keeping optional extensions separate.
R-03	Security or privacy breach exposing personal data, financial household information, private messages, automation records, or command-parser logs.	Medium	Critical	Use secure authentication, password hashing, HTTPS in deployment, RBAC, input validation, and least-privilege access. Avoid logging sensitive user content, tokens, raw audio, and unnecessary LLM prompts.
R-04	Incorrect expense calculation or settlement logic leading to wrong balances between roommates.	Medium	Critical	Implement the settlement engine separately from the UI. Use decimal data types instead of floating-point arithmetic. Create unit tests for equal splits, unequal splits, rounding, deleted users, and partially settled payments.

R-05	GDPR compliance gaps because the system processes personal data such as names, emails, tasks, expenses, messages, usage history, command text, LLM parser output, and automation commands.	Medium	High	Maintain a data inventory, define clear purposes for each data type, minimize collected data, provide deletion or anonymization options, and document external processors if hosting or external AI services are used.
R-06	NFC functionality does not work reliably because active Web NFC support is limited across browsers and devices.	High	Medium	Use passive NDEF URL routing where supported and treat NFC appliance logging as optional extended functionality so it does not block the core system. Build an early prototype before investing major effort.
R-07	Integration problems between Angular frontend, Spring Boot backend, H2 database, Whisper-Flow service, and Google Cloud Vertex AI Gemini integration.	Medium	High	Define REST/WebSocket contracts early, use consistent DTOs, provide sample data, and run integration tests in the CI/CD pipeline. Use containerized local setup where applicable.
R-08	Data loss due to faulty database initialization, migrations, accidental deletion, or development mistakes.	Low	Critical	Version-control database initialization/migration files, use test data for development, define a backup/export strategy for demo data, and test restore procedures before final delivery.
R-09	Performance problems when dashboards, expense histories, chats, feeds, automation logs, or command-parser logs contain many records.	Medium	Medium	Use pagination, database indexes for frequently queried fields, and avoid loading all historical data at once. Include performance smoke tests for dashboard and history pages.
R-10	Team availability risk due to uneven workload, illness, exams, or unclear responsibilities.	Medium	High	Use primary and backup roles, weekly progress checks, visible task boards, and early escalation of blockers. Critical features should not depend on only one team member.

R-11	External hosting restrictions or vendor lock-in.	Medium	Medium	Prefer university-provided or EU/EEA-based hosting where possible. Keep the application containerized so it can be moved between environments. Document all external services and avoid unnecessary third-party dependencies.
R-12	Notification, chat, or automation features create unwanted spam or social pressure between roommates.	Low	Medium	Allow users to configure notification preferences. Keep reminders factual and task-based. Avoid public exposure of private WG information by default.
R-13	Voice, ASR, or LLM automation misinterprets user commands and creates incorrect expenses, shopping updates, task changes, or appliance logs.	Medium	High	Restrict automation to predefined intents, validate JSON outputs against a schema, show a confirmation dialog before sensitive actions, and log executed automation actions for traceability.
R-14	Real-time speech transcription introduces high latency or fails to recognize specific WG vocabulary.	Medium	Medium	Use a standalone Whisper-Flow container optimized for streaming, provide manual text input fallback, and allow users to edit transcribed text before confirmation.
R-15	Optimization chaos caused by trying to support too many devices, browsers, screen sizes, and operating system versions.	Medium	High	Define a strict compatibility matrix. Optimize primarily for iPhone 12 Pro with iOS 26.x/Safari, Google Pixel 7 with Android 16/Chrome 147.x+, and Chrome 147.x or Firefox Stable at 1366x768 px or higher. Other devices are supported only on a best-effort basis.
R-16	LLM prompt injection or malformed model output causes unintended behavior or unsafe automation proposals.	Medium	High	Treat all LLM output as untrusted input. Use a strict JSON schema, whitelist allowed intents, reject unknown fields, validate all entity references on the backend, and require confirmation before sensitive actions.

R-17	Google Cloud Vertex AI integration is blocked by quota, billing, network, or credential issues.	Medium	Medium	Keep written natural-language input available and keep the LLM call backend-only and configurable. If Google Cloud access, billing, quota, or network permissions cause problems, the integration can be demonstrated with documented sample parser outputs while the core system remains functional.

8.2 Quality Assurance Strategy

Quality assurance is integrated into the development process rather than treated as a final project phase. The main goal is to detect defects early, especially in security-related workflows, data consistency, expense calculation, LLM command parsing, and automation handling.

Testing levels:

- **Unit tests:** Used for backend services, especially expense splitting, payment settlement, task assignment, invitation handling, notification logic, JSON schema validation, and automation validation.
- **Integration tests:** Used to verify the interaction between the REST API, database, authentication layer, Whisper-Flow service, Google Cloud Vertex AI Gemini integration, and important business workflows where implemented.
- **Frontend tests:** Used for important UI workflows such as registration, adding expenses, completing tasks, viewing the dashboard, and confirming automation actions.
- **Manual acceptance tests:** Used before each milestone to check whether the implemented features solve the intended WG use cases.
- **Reference-device tests:** Used to verify that core workflows remain usable on iPhone 12 Pro/iOS 26.x/Safari, Google Pixel 7/Android 16/Chrome 147.x+, and desktop browsers.

Review process:

- Every major feature should be implemented on a separate Git branch.
- A feature should only be merged after another team member has reviewed the code.
- Critical logic, especially authentication, settlement calculation, command parsing, LLM prompt handling, and automation execution, should receive additional review by the QA/Risk responsible person or the technical architect.
- The CI/CD pipeline should run automated tests before code is merged into the main branch.

Acceptance criteria: A feature is considered complete only if it is implemented, tested, reviewed, documented where necessary, and integrated into the current system version without breaking existing functionality.

8.3 Information Process

The team follows a simple communication and documentation workflow to ensure smooth coordination during development.

Communication:

- WhatsApp is used for daily communication and quick questions.
- Weekly meetings are held to review progress, discuss blockers, and assign new tasks.
- Additional meetings are scheduled when urgent issues arise.

Development Workflow:

- GitLab is used for source code management with stable code on `main`.
- Each team member works on feature branches named `feature/FR-ID-short-name`; bug fixes use `bugfix/short-description`.
- Features are reviewed, tested, and merged only after the CI/CD pipeline passes.
- Integration happens continuously throughout the project, but extended features should not destabilize the core workflow.

Documentation:

- Shared documents are used for requirements, diagrams, meeting notes, and reports.
- Documentation is updated whenever major changes are made.
- Known limitations of optional features should be documented clearly before final submission.

9 Legal, Compliance and External Hosting Constraints

WGMS processes personal data because users create accounts, join households, receive tasks, record expenses, send messages, and may generate appliance usage, ASR transcripts, LLM parser outputs, or automation records. Therefore, even though the project is developed in a university context, privacy and data protection requirements must be considered from the beginning.

9.1 Processed Data Categories

The system may process the following categories of data:

- **Account data:** Name, email address, password hash, authentication tokens, and user role.
- **WG membership data:** Household membership, invitation links, user roles such as Main Tenant or Roommate, and join/leave timestamps.
- **Task data:** Assigned chores, completion status, due dates, and task history.
- **Expense data:** Shared expenses, paid amounts, participants, payment status, and settlement history.
- **Shopping data:** Shopping items, requests, suggestions, votes, and generated expense references.
- **Communication data:** Notifications, direct messages, group messages, WG conversations, feed posts, and event-related messages.
- **Appliance usage data:** Appliance identifiers, NFC scan records, usage timestamps, and the user who created the usage record.
- **Voice and automation data:** Temporary audio streams, transcribed commands, parsed intents, structured JSON payloads, confirmation decisions, execution status, and timestamps.
- **LLM parser data:** User command text, allowed-intent prompts, limited context snapshots for entity matching, parser output, confidence value, validation result, and user-facing summary.
- **Technical data:** Application logs, error messages, timestamps, and security-relevant events.

The project does **not** require processing of special categories of personal data such as health data, religious beliefs, political opinions, or biometric data. The system also does **not** execute real money transfers and does not need to store bank account numbers, credit card data, or payment provider credentials. Raw audio data should only be processed temporarily for transcription and should not be stored permanently.

9.2 GDPR-Oriented Design Principles

The design of WGMS follows the most important GDPR-oriented principles:

- **Purpose limitation:** Data is collected only for clear WG management purposes such as organizing chores, expenses, shopping, communication, shared resources, and confirmed automation actions.
- **Data minimization:** Only data needed for the planned features is stored. Optional features such as public feed posts, NFC appliance tracking, cross-WG communication, voice automation, or LLM parsing should not be activated by default if they are not necessary for the core WG workflow.

- **Transparency:** Users should be informed which data is stored and why. A short privacy notice should be included in the final documentation or application.
- **Storage limitation:** Data should not be kept longer than necessary. Invitation tokens should expire automatically. Deleted accounts should be removed or anonymized where possible.
- **Integrity and confidentiality:** Personal data must be protected through authentication, authorization, secure password handling, HTTPS in deployment, restricted database access, careful log handling, and minimal LLM context sharing.
- **Accountability:** The team documents the main data categories, security measures, external services, AI/LLM components, and known privacy risks.

9.3 User Rights and Data Handling

The system should support basic user rights as far as possible within the project scope:

- **Access:** A user should be able to view their account information, WG membership, assigned tasks, expenses, communication records, appliance usage records, automation history, and stored command-parser records where relevant.
- **Correction:** Incorrect profile, household, task, shopping, expense, or automation data should be editable by the user or by an authorized WG administrator.
- **Deletion:** Users should be able to leave a WG and request account deletion. Where financial history must remain understandable for other roommates, the user can be anonymized instead of fully deleting historical expense records.
- **Export:** If feasible, the system should provide a simple export of user-related data or allow manual database export during the project demo.

9.4 External Hosting Constraints

If WGMS is deployed externally, the following constraints apply:

- Hosting should preferably take place on university-provided infrastructure or servers located in the EU/EEA.
- If a third-party hosting provider or external AI provider is used, the provider should be documented as an external processor.
- The LLM parser uses Google Cloud Vertex AI / Gemini as an external AI service. The provider, region, service account, model ID, prompt contents, and processed data categories should be documented.
- If personal data is transferred outside the EU/EEA, the team should check whether appropriate safeguards such as adequacy decisions or standard contractual clauses are required.
- Production secrets, database passwords, JWT secrets, and API keys must not be committed to Git.
- Test environments should use artificial demo data instead of real personal data whenever possible.
- Application logs should not contain passwords, tokens, private messages, raw audio, full financial details, or unnecessary LLM prompts/responses.

9.5 Security Constraints

Security is especially important because the system combines personal communication, financial household records, automation commands, LLM parsing, and role-based WG administration. Therefore, the following constraints are defined:

- Passwords must be hashed and salted.
- Authentication tokens must expire after a reasonable time.
- Invitation links must be time-limited and difficult to guess.
- Users may only access data belonging to their own WG unless a feature explicitly supports public cross-WG interaction.
- Main Tenant or administrator functions must be protected by role checks on the backend, not only hidden in the frontend.
- NFC-based appliance usage must be transparent to users and should not be used for hidden monitoring.
- Automation actions are validated on the backend before execution.
- Sensitive automation actions should require explicit user confirmation.
- LLM output must always be treated as untrusted input.
- The LLM parser should not receive authentication tokens, password hashes, private message histories, or raw audio files.
- The Google Cloud Vertex AI integration should use restricted service-account credentials, least-privilege IAM roles, and protected backend-only API access. Frontend clients should not call the Gemini API directly.

9.6 Residual Compliance Risk

The main residual compliance risk is that WGMS can become more privacy-sensitive if optional features such as public WG feeds, cross-WG communication, appliance usage tracking, voice-assisted automation, or LLM-based command parsing are expanded. These features increase the amount of behavioral, social, and inferred action data stored by the system. Therefore, they should remain optional, limited in scope, and clearly separated from the core private WG management functions.

For the current project scope, no real payment execution, no banking integration, no persistent raw audio storage, a documented external Google Cloud Vertex AI / Gemini dependency for command parsing, and no special-category personal data are planned. This keeps the compliance risk manageable and appropriate for a university software engineering project.